File-Level Is Not Symbol-Level: What Code Graphs Miss and the Issue Already States

300 tasks, then 2,232: a file-level hit is 16× the text of the fix and one of 48 definitions; the graph adds two points, and what the pool never reaches never names code

Abstract

An agent reads characters, not slots, yet the retrieval that feeds it is reported at file granularity. A file-level hit at the median Lite task hands over 29 KB — 16× the 1.7 KB definition the fix touches, one of 48 in that file — and the metric counts the hit. On 300 SWE-bench Lite tasks we extract the **354 symbols the reference patches actually touch**: the same ranking that recovers **62.4%** of gold files at rank 10 recovers **27.5%** of gold symbols, a 2.3× gap that is 2.28× again on 2,232 tasks — **1.97×** once the documents include the body. **Eight ways of adding a code graph leave symbol-level recall unchanged** on 300 tasks and buy 2.2 points on 2,232, none at file level. The signal pays, not the structure: sixteen cheap features reach **33.2–36.1%** of symbols, a 33M bi-encoder matches them, and together they reach **44.7%** of symbols and **72.7%** of files; the 27.6% the pool never reaches is 85% unnamed. We release the benchmark and the harness.

1. Introduction

An agent that cannot see a repository must search it: the context holds a few hundred symbols, the repository tens of thousands, and everything downstream depends on the right one being among the few.

LocAgent reports 92.7% file-level accuracy on SWE-bench-style tasks [2] — but a large module holds dozens of definitions, and opening the right file still leaves the agent to find the one function the fix belongs in.

We measure the fine granularity. For each of 300 SWE-bench Lite tasks the reference patch gives the symbols the fix touched, and the same ranking that finds 62.4% of gold files at rank 10 finds 27.5% of gold symbols. **The gap is 2.3×, and it is not an artefact — it is one list, scored two ways.**

One instance: `matplotlib__matplotlib-24970`'s traceback names file and line, so every arm puts `lib/matplotlib/colors.py` in its top five; not one puts `Colormap.__call__` in its top ten, though the pool contains it.

Second, we test the standard remedy: build a call graph and let structure carry the rest. None of eight arms improves symbol-level recall at ten slots — not even personalised PageRank — and the most aggressive costs 4.8 points of file-level recall ($p = 0.004$). What does gain is the source the issue quotes (§5).

The explanation is arithmetic: one-hop balls are 0.42% gold against 3.1% per text slot.

2. Related Work

Repository-level benchmarks. SWE-bench and its subsets score whether a patch makes the failing tests pass [1], and say nothing about whether the code could be found. Our targets derive from its patches, but symbol-level retrieval is orthogonal to resolved rate.

Agents over repositories. SWE-agent fixes the interface and lets the model act [5]; LocAgent navigates with graph structure [2]; ARISE builds a repository graph and lifts Function R@1 from 0.43 to 0.60 [3]. These systems co-design retrieval with a model, so their numbers are not comparable: the graph's marginal value is the only thing that varies.

Granularity in 2026. CORE-Bench [8] shows that repository search is not docstring matching, RepoNav [9] that flat snippet lists hide the target function, and FuseSearch [10] drops the code graph entirely while still reporting file-level F1. None of them prices the structure; that is what we do, at two scales.

Code graphs. PyCG reports 99.2% precision and 69.9% recall on its own benchmark [4]; CodexGraph turns a repository into a queryable code graph [6], and RepoAtlas maintains evolving repository views for agents [7]. We report ours — 46% of call sites definite — and the ceiling that bounds one-hop expansion.

3. Setup

Tasks. All 300 SWE-bench Lite instances, 12 repositories, dominated by django (114) and sympy (77); 297 distinct base commits as tarballs (2.5 GB). Nothing is executed: the measurement is over source.

Gold symbols. For each task we parse the reference patch and map every changed line to the innermost definition containing it in the *pre-fix* tree: a deleted line anchors itself, an added line the line before it, since the added text does not exist before the fix. That gives 354 definitions over 290 tasks; the other 10 are dropped: 2 change only module-level lines, 8 only add definitions. We record rather than drop the 87 changed lines outside any definition, so hard cases cannot inflate the score. A symbol is `<module>.<qualname>`, module being the file path without `.py`; file-level metrics group the same ranking by file. A second implementation reproduces 279 of 300 keys; rescoring moves recall by at most 0.7 points.

Query and budget. The issue text as given: no rewrite, no file hints, no LLM preprocessing — the weakest defensible query. Every arm fills exactly 10 slots, so an arm cannot win by retrieving more.

4. Method

The graph builder. We build a definition-and-reference graph from source with Python's `ast`, in the same JSON shape as the competition's released assets, so one metric applies to both. Nodes are modules, classes, functions and methods, each carrying its signature, docstring, and source span; edges come in four types — `defines`, `imports`, `inherits`, `calls`.

Resolution states. What matters is not the edge list but what each edge means. We record seven states and never collapse them:

state	meaning
`local`	target defined in the same module
`import`	target reached through an import alias
`unique`	leaf name occurs exactly once in the repository
`suffix`	target matched a dotted suffix (src/ layouts)
`external`	stdlib or third-party; not a defect
`ambiguous`	more than 64 same-named candidates; refused rather than guessed
`unresolved`	no target found

On a django snapshot (2,715 files, 38,651 nodes, 209,502 edges, 10.8 s) the distribution is 34.1% unresolved, 19.2% local, 15.6% import, 15.0% ambiguous, 11.2% unique, 4.9% external. **Only 46% of call sites resolve to a definite in-repository target** — the quality of the artefact the arms expand, and the reason `graph_fuse_precise` exists.

The arms. Eight, all paired by task; `text` is BM25 over a per-symbol document (path, qualified name, signature, docstring), and the other seven fold in the graph. `graph_rerank` promotes anything one hop from the top five seeds without changing the pool, so it cannot add a candidate. `graph_open` fills slots 6–10 with neighbours of the top five — the only arm that can add a symbol BM25 never ranked. `graph_fuse` lets a candidate inherit $\lambda = 0.5$ of its best adjacent seed's score, **fixed before the run and never tuned**; `graph_fuse_precise` fuses over only the definite 46%. `graph_ppr` is personalised PageRank over the whole graph (restart 0.15, the convention, not tuned); `graph_ppr_text` adds half the diffusion mass to normalised BM25. Neighbours are unioned over `calls` and `inherits` and drawn round-robin so a hub cannot absorb the shortlist.

Two more rankings. The evidence arm reads what the issue states: a traceback frame resolves through the same symbol table we built, a named path is a suffix match, and a quoted line is found by indexing the candidate's *body* — text our symbol documents never contain. The learned arm puts sixteen signals into one ranker (§5).

Statistics. Per-task differences, so arms are compared on the same issue and repository: mean difference with a bootstrap 95% interval (2,000 resamples, fixed seed) plus an exact sign test, which decides because ties dominate.

5. Results

The granularity gap is the headline. Table 1, one ranking, two ways of scoring it. All 354 gold symbols resolve into the graph, so the denominator is not in question.

***Table 1** — symbol vs file granularity (n = 290 scored tasks)*

granularity	recall@1	recall@5	recall@10
symbol	0.091	0.214	**0.275**
file	—	0.548	**0.624**

Line granularity agrees: the ranking covers **23.9%** of the 1,010 changed lines, so the gap is not a matching artefact. **At ten slots, no graph arm improves symbol-level recall** — Table 2 and Figure 1.

The file costs sixteen times the fix. At the median Lite task the gold file is 28,950 characters and the definition the patch touches is 1,726 — 6.2% of the file, one of 48; 112 of 290 tasks exceed 25×. A file-level hit buys the agent the search, not the answer.

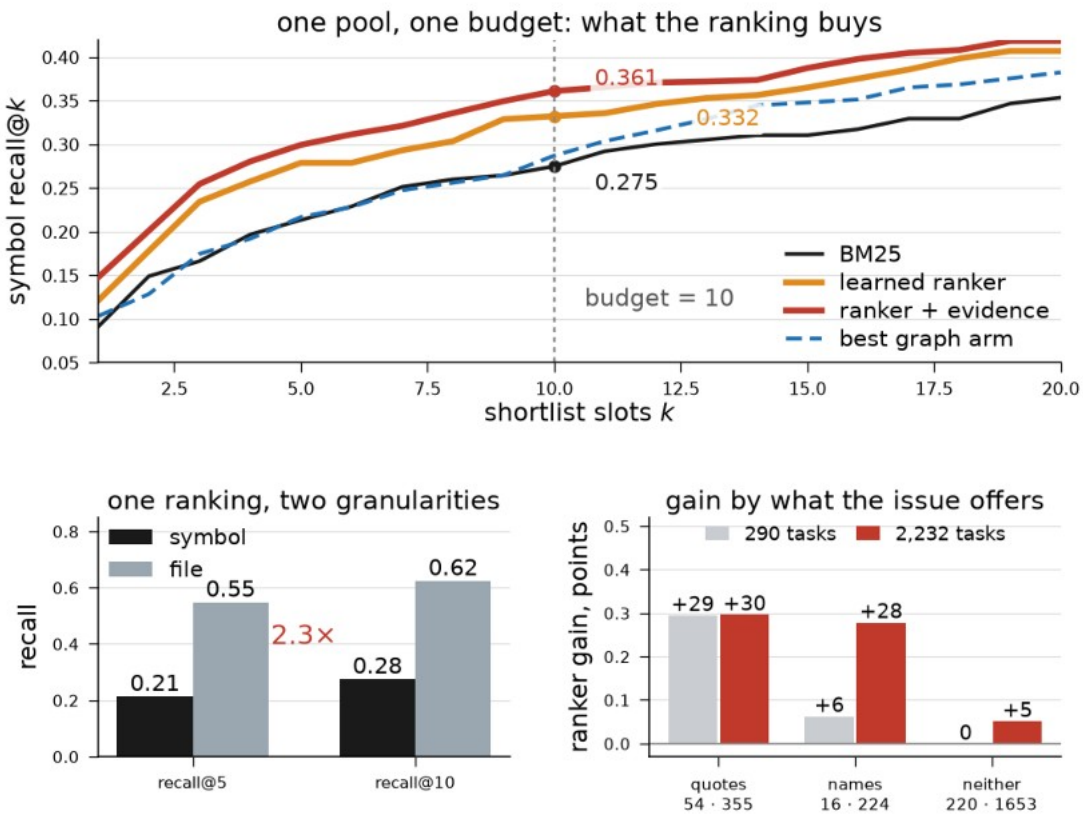


***Figure 1** — top: symbol recall@k for BM25, the best graph arm, the learned ranker and the ranker with evidence. Bottom left: the same ranking by file and by symbol. Bottom right: the ranker's gain over BM25 by what the issue offers, at both scales.*

***Table 2** — the eight arms of §4, paired, equal pool and equal character budget*

arm	r@1	r@5	r@10	file r@10	Δr@10 (95% CI)	p
text	0.091	0.214	0.275	0.624	—	—
graph_rerank	0.103	0.217	0.287	0.614	+0.012 (−0.026, 0.050)	0.50
graph_open, 1 hop	0.091	0.214	0.257	0.576	−0.018 (−0.051, 0.013)	0.46
graph_open, 2 hops	0.091	0.214	0.249	0.579	−0.026 (−0.057, 0.003)	0.18
graph_fuse	0.091	0.217	0.272	0.624	−0.003 (−0.010, 0.000)	1.00
graph_fuse_precise	0.091	0.217	0.275	0.624	0.000	1.00
graph_ppr	**0.106**	0.214	0.268	0.579	−0.007 (−0.040, 0.027)	0.72
graph_ppr_text	0.091	0.217	0.272	0.624	−0.003 (−0.010, 0.000)	1.00

The second-hop variant reaches furthest (35.7%, Table 3) and is still negative. Reranking survives a wider budget: +2.9 at k = 20 (13 wins, 3 losses, p = 0.021).

Diffusion is a lateral move; expansion is worse. Personalised PageRank wins the first slot (0.106 against 0.091) and still trades 15 gained gold symbols for 17 lost; at file level it loses 4.5 points (p = 0.019). `graph_open` swaps slots 6–10 for neighbours: 18 tasks lose a gold file, 4 gain one, −4.8 points (p = 0.004). **Reach bounds the arm; the exchange rate eats the bound** — Table 3.

***Table 3** — diagnostics, ceilings and the ablation grid (n = 290)*

quantity	value	meaning
reach, 2 hops, 1 / 3 / 5 / 10 seeds	0.191 / 0.310 / 0.357 / 0.395	radius buys reach quickly
one-hop ball, median size / gold density	21 nodes / 0.42%	41 gold symbols among 9,855 neighbours
diffusion's conversion of the ball	18 / 41 (44%)	reachable gold placed in its top 10
oracle@10, pool only / pool plus neighbours	0.600 / 0.630	a perfect reranker's ceiling, with and without expansion
tasks whose gold symbols never enter the top-500 pool	80 / 290 (27.6%)	the hard tail

One-hop expansion has an arithmetic ceiling: 38 of 290 tasks (13.1%; 41 of 354 symbols) hold a gold symbol one hop from a top-five seed, so perfect conversion is worth 11.9 points; two hops lift the ceiling to 35.7%. Conversion is not the failure — diffusion places 18 of the 41 reachable symbols in its top ten (44%) and still lands below text, because 15 gained cost 17 lost. The rate is the finding: 9,855 neighbours hold 41 answers, 0.42%, against 3.1% gold per text slot.

Where the gap is. Ten slots are a budget: recall@10 ≈ P(gold file selected) × E[recovered | depth]. Given the *right* file they recover **66.5%** of gold symbols.

A learned ranker. Sixteen features — lexical score and rank, body-quote overlap, path and frame matches, four graph statistics, size and shape terms — feed a LightGBM ranker trained on six repositories, tested on the other six, three seeds averaged, nothing tuned on test.

***Table 4** — learned ranking and evidence (n = 290)*

ranking	r@10	Δ vs BM25	W/L	p
BM25	0.275	—	—	—
learned ranker, 16 features	0.332	+0.057 (0.019, 0.096)	31/14	0.016
+ evidence, 5 slots	**0.361**	**+0.086 (0.046, 0.126)**	**39/13**	**<0.001**

The learned combination is worth 5.7 points on repositories it has never seen.

A dense score is complementary. A 33M bi-encoder (bge-small-en-v1.5) scoring the same 200 candidates by cosine reaches 36.6% of symbols and 69.3% of files alone; as one more column it takes the ranker to **44.7%** and **72.7%** (+8.6, 418 wins to 83 losses). The gap narrows with retriever strength — 2.28×, 1.89×, 1.78×, **1.63×** — and does not close.

***Table 5** — the ranker split by what the issue offers, at both scales (left 290 tasks, right 2,232)*

subset	n	BM25	ranker	n	BM25	ranker
quotes the gold body	54	0.481	**0.776**	355	0.459	**0.755**
names a path or a frame	16	0.406	0.469	224	0.380	**0.659**
neither	220	0.215	0.214	1,653	0.185	0.236

On tasks whose issue pastes a line from the gold body the ranker gains **29.4 points at 290 tasks and 29.6 at 2,232**. The middle bucket: naming a file or a frame looked worth 6 points on sixteen tasks and 27.8 on 224. What the issue *states* is the signal; the graph is not.

The same claims on 2,232 tasks. Table 6 repeats them on the full split — 2,174 snapshots, same code, same denominators.

***Table 6** — what survives a 7.7× larger sample*

reading	290 tasks	2,232 tasks
file / symbol recall@10	0.624 / 0.275 = 2.27×	0.564 / 0.248 = **2.28×**
best graph arm, symbol	+0.012 (−0.026, 0.050)	+0.022 (0.013, 0.031)
best graph arm, file	−0.010	+0.000 (−0.009, 0.008)
learned ranker, symbol	+0.057	+0.113 (0.098, 0.128)
learned ranker, file	—	0.564 → 0.643
graph features, with − without	−0.017, p = 0.50	+0.027 (0.017, 0.036)
evidence override over the ranker	+0.029, p = 0.035	−0.002 (−0.008, 0.004)

Three things move and one does not. The **gap is invariant to sample size**, 2.27× against 2.28×. The **graph is worth two points at symbol level and zero at file level**: invisible at 290 tasks, +0.022 at 2,232 with an interval excluding zero, 169 wins to 90, positive in 8 of 12 repositories — small, because it reorders symbols inside files text already found. The **evidence override does not survive**: at full scale the ranker reaches 0.361 alone and the override costs it 0.2 points at the same five-slot cap; 2,232 tasks let the ranker learn the reorder, 290 did not.

What you index moves the gap. Adding the candidate's body raises symbol recall at 2,232 tasks from 0.248 to 0.286 (+3.8, CI 2.1–5.4): the gap narrows from 2.28× to 1.97×. Another symbol's body instead drops recall to 0.128, so the gain is the candidate's own text; as two ranker columns it is worth **+2.6 points** (202/94).

How many tasks a claim needs. Resampling the 2,232 paired differences and running the same sign test — 400 draws per cell — gives the share of n-task studies that would call an effect real: +11.3 points is detected at n = 100 in 88% of draws, +2.2 in 39% at 290 and 87% at 800, −0.2 never.

***Table 7** — share of resamples whose sign test reaches p < 0.05, by sample size*

effect, full scale	n = 100	290	500	800	2,232
ranker, +11.3 points	0.88	1.00	1.00	1.00	1.00
graph, +2.2 points	0.09	0.39	0.62	0.87	1.00
evidence, −0.2 points	0.01	0.02	0.03	0.04	0.00

Reproducibility. Every number is re-derived from the released JSONL by a 259-assertion check: a claim that drifts from the data fails the build, not the review.

6. Discussion

The practical claim is a trade — 2.3× harder to hit, 16× cheaper when it does — and expansion is not where the error is (88.1% of gold symbols sit beyond one hop); the margin is what an issue states. The 27.6% whose gold never enters a top-500 pool bounds that margin: on those 80, the issue quotes the gold body once, names its file once, shows a frame naming it eight times, and names the symbol five — 15% in all. The other 68 describe behaviour and never name code, and a dense index over every definition fails the same way: union with the pool covers 62.8% of Lite tasks, the lexical rate.

The measurement claim is granularity: file-level numbers near 90% are quoted as evidence the hard part is over; the same ranking is 27.5% on symbols.

A stronger reranker would not overturn it: a perfect one reaches 0.600, and the graph's headroom is three points. What would is a pool drawn by behaviour, or a graph built by type inference [4].

7. Limitations

Python only, as are most tasks. The eight-arm comparison uses one retriever, so the graph result is about its marginal value *given this ranking*; the ranker trains on six repositories and is tested on six. The evidence channel does nothing on the 74% naming no source, and pytest loses 12.5 points to test-file tracebacks. We measure retrieval, not resolution: a better shortlist is necessary for a better patch, not sufficient.

8. Conclusion

Same ranking, two granularities: 62.4% of gold files, 27.5% of gold symbols. Eight graph arms, none improving symbol-level recall and three hurting file-level recall; on 2,232 tasks the best is worth two symbol points and zero file ones. The arithmetic explains why: an 11.9-point reach ceiling, a ball that is 0.42% gold against 3.1% for a text slot, and three points of graph headroom. The same ten slots reach **33.2–36.1%** once a ranker reads what the issue states — which is the difference between inferring structure and reading it.

References

[1] Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024.

[2] Chen et al. LocAgent: Graph-Guided LLM Agents for Code Localization. 2025.

[3] Seddik et al. ARISE: A Repository-level Graph Representation and Toolset for Agentic Program Repair and Fault Localization. 2026.

[4] Salis et al. PyCG: Practical Call Graph Generation in Python. ICSE 2021.

[5] Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024.

[6] Liu et al. CodexGraph: Bridging Large Language Models and Code Repositories via Code Graph Databases. NAACL 2025.

[7] Zhang et al. RepoAtlas: Guiding Coding Agents via Evolving Multimodal Repository Views. 2026.

[8] Zhang et al. CORE-Bench: A Comprehensive Benchmark for Code Retrieval in the Era of Agentic Coding. EMNLP 2026.

[9] Chai et al. RepoNav: From Snippet Retrieval to File-Centered Repository Navigation for Code Agents. EMNLP 2026.

[10] FuseSearch: Learning Adaptive Parallel Execution for Efficient Code Localization. ACL 2026 Findings.